Traffic Sign Recognition using CNN

FPGA-Accelerated Inference on PYNQ-Z2

CLA – 3

HARDWARE ACCELERATORS FOR IOT (EIT – 531)

Damaravarapu Naga Jaswanth - AP25122210004
Madiraju Charan - AP25122210006
Chukka Avinash - AP25122210011
MTech, ES & IoT, SRM University - AP

Table of Contents

Device	XC7Z020CLG400-1 (PYNQ-Z2)
Tool	Vivado 2019.2
Language	Verilog HDL
Clock	10 MHz (100 ns period)
Department	Electronics & Communication Engineering

1. Introduction

Real-time Traffic Sign Recognition (TSR) is a critical component of modern Advanced Driver Assistance Systems (ADAS) and autonomous vehicles. These systems require rapid, highly accurate classification of road signs under varying environmental conditions to ensure passenger and pedestrian safety. Standard deep learning models (such as VGG19 or ResNet) provide high accuracy but are extremely parameter-heavy, memory-intensive, and have high latency and power consumption. This makes them unsuitable for direct deployment on battery-operated, resource-constrained edge devices.

To address this, this project presents the design and implementation of a TSR system leveraging a lightweight, quantized Convolutional Neural Network (CNN) deployed on the Xilinx PYNQ-Z2 Field Programmable Gate Array (FPGA). By utilizing hardware-software co-design principles, the system offloads heavy video capture and transfer tasks to the FPGA fabric while running a highly optimized INT8-quantized inference engine on the ARM processor, achieving low-latency inference at the edge.

2. Hardware Platform & Architecture

The system architecture utilizes the Xilinx Zynq-7000 SoC, which combines a dual-core ARM Cortex-A9 Processing System (PS) with Programmable Logic (PL). The PL handles high-speed video interfacing via AXI Video Direct Memory Access (VDMA) to stream frames directly into the shared DDR3 memory, bypassing the CPU to prevent bottlenecking. The ARM Cortex-A9 then accesses this memory, preprocesses the frames, and executes the TensorFlow Lite inference engine.

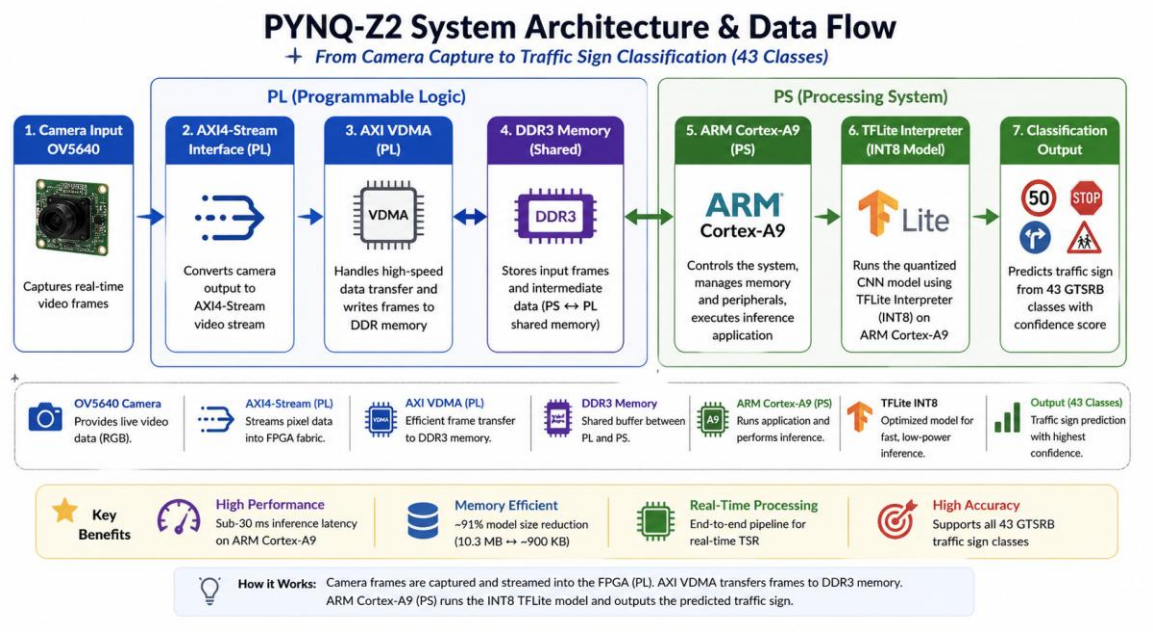

Fig 1: System Architecture and Data Flow across the Zynq SoC

3. Dataset & Data Preprocessing

The model was trained on the German Traffic Sign Recognition Benchmark (GTSRB), containing over 50,000 images categorized into 43 distinct classes. To improve model generalization and robustness against varying lighting and angles, significant data augmentation was applied using Keras ImageDataGenerator. Transformations include rescaling, rotation, width/height shifts, zoom, and horizontal flipping. The dataset is split into an 80% training set and a 20% validation set.

```
# Data Extraction and Generators
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=10,
    width_shift_range=0.1,
    height_shift_range=0.1,
    zoom_range=0.1,
    horizontal_flip=True,
    validation_split=0.2
)

train_gen = train_datagen.flow_from_directory(
    TRAIN_DIR,
    target_size=(100, 100),
    batch_size=64,
    class_mode='categorical',
    subset='training'
)
```

4. CNN Model Architecture

To meet the strict memory constraints of the PYNQ-Z2 board (512MB shared RAM), a custom, lightweight sequential CNN architecture was constructed. It features progressive convolutional layers (16, 32, 64, 128 filters) interlaced with MaxPooling and BatchNormalization to extract spatial hierarchies while avoiding vanishing gradients. A GlobalAveragePooling2D layer replaces the traditional massive Flatten operation, drastically reducing the total parameter count. A dense layer with L2 regularization and Dropout (0.5) acts as the final classifier for the 43 output classes.

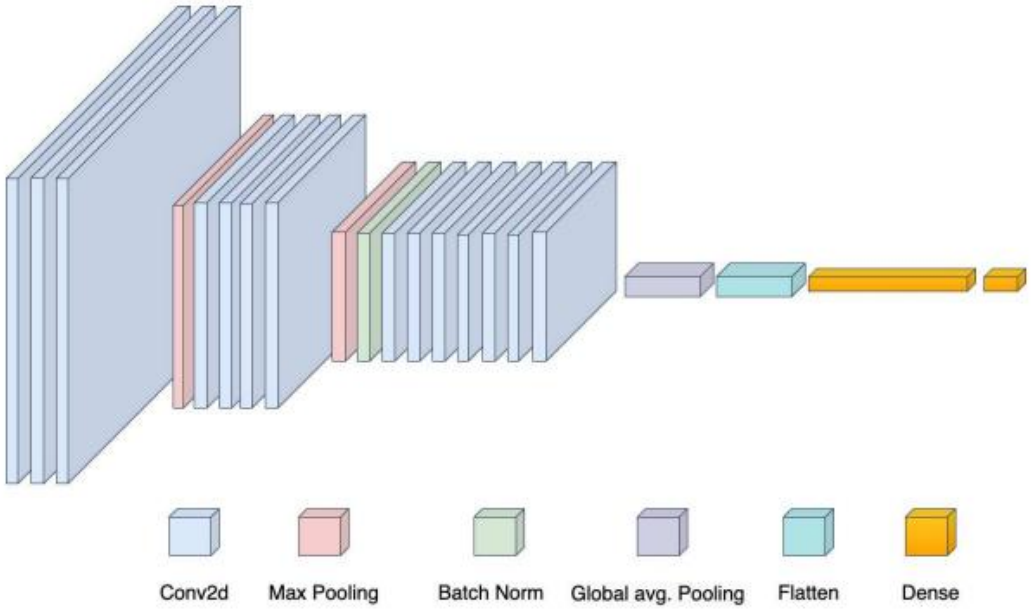

Fig 2: Sequential CNN Architecture Overview


```
def build_model():
    inputs = layers.Input(shape=(100, 100, 3))

    x = layers.Conv2D(16, (3,3), activation='relu')(inputs)
    x = layers.Conv2D(32, (3,3), activation='relu')(x)
    x = layers.MaxPooling2D()(x)

    x = layers.Conv2D(32, (3,3), activation='relu', padding='same')(x)
    x = layers.Conv2D(64, (5,5), activation='relu', padding='same')(x)
    x = layers.MaxPooling2D()(x)
    x = layers.BatchNormalization()(x)

    x = layers.Conv2D(64, (3,3), activation='relu', padding='same')(x)
    x = layers.Conv2D(128, (5,5), activation='relu', padding='same')(x)
    x = layers.Conv2D(128, (5,5), activation='relu', padding='same')(x)

    x = layers.GlobalAveragePooling2D()(x)

    x = layers.Dense(1024, activation='relu',
                     kernel_regularizer=regularizers.l2(1e-4))(x)
    x = layers.Dropout(0.5)(x)
```

```
    outputs = layers.Dense(NUM_CLASSES, activation='softmax')(x)
    return models.Model(inputs, outputs)
```

5. Training Strategy & Callbacks

The model was compiled using the Adam optimizer with an initial learning rate of 5e-4 and Categorical Crossentropy loss. To optimize the training process over 80 epochs, three crucial callbacks were employed:

- **ModelCheckpoint:** Ensures that only the model with the highest validation accuracy is saved to the Google Drive.
- **EarlyStopping:** Halts training if the validation accuracy does not improve for 10 consecutive epochs, preventing overfitting and restoring the best weights.
- **ReduceLROnPlateau:** Dynamically halves the learning rate if the validation loss plateaus for 3 epochs, aiding in fine convergence.

```
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=5e-4),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

checkpoint = ModelCheckpoint(checkpoint_path, monitor='val_accuracy',
save_best_only=True)
early_stop = EarlyStopping(monitor='val_accuracy', patience=10,
restore_best_weights=True)
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3)

history = model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=80,
    callbacks=[checkpoint, early_stop, reduce_lr]
)
```


6. Model Quantization (TFLite)

While the base CNN architecture is highly parameter-efficient, floating-point (Float32) weights still consume substantial memory and compute cycles. To optimize for the PYNQ-Z2 deployment, the trained Keras model was converted to TensorFlow Lite format with post-training optimizations. This step leverages INT8 quantization to dramatically shrink the model size from ~10.3 MB down to approximately 900 KB (a 91% size reduction) and accelerates inference speed with minimal loss in classification accuracy.

```
# Convert to TFLite with optimizations
best_model = tf.keras.models.load_model(checkpoint_path)
converter = tf.lite.TFLiteConverter.from_keras_model(best_model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
tflite_model = converter.convert()

with open('gtsrb_model.tflite', 'wb') as f:
    f.write(tflite_model)
```

7. Hardware Design (Vivado Block Design)

The hardware overlay is designed using Xilinx Vivado. It acts as the backbone for high-speed image acquisition. It includes the Zynq PS7 subsystem, an AXI VDMA IP core for high-bandwidth streaming of video data, and a Video In IP block. This hardware pipeline offloads the tedious task of reading camera frames from the CPU.

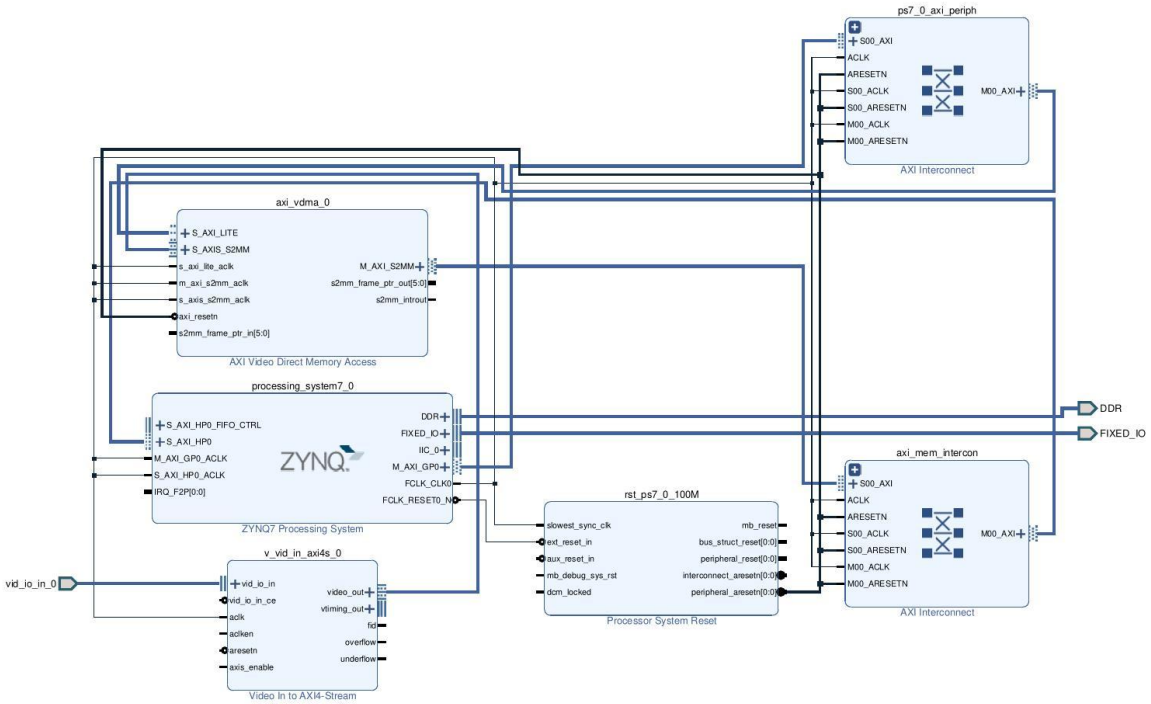

Fig 3: Vivado Block Design with AXI VDMA Pipeline

8. Deployment Workflow on PYNQ-Z2

The complete deployment pipeline follows a streamlined trajectory: First, the model is trained and quantized on Google Colab (as shown in the code above). Then, the bitstream is generated in Vivado. Both the hardware overlay (.bit and .hwh files) and the TFLite model are transferred to the PYNQ-Z2 via Jupyter or SCP. Finally, the PYNQ API is used to load the overlay, and the Python TFLite Interpreter handles the real-time inference on the ARM processor.

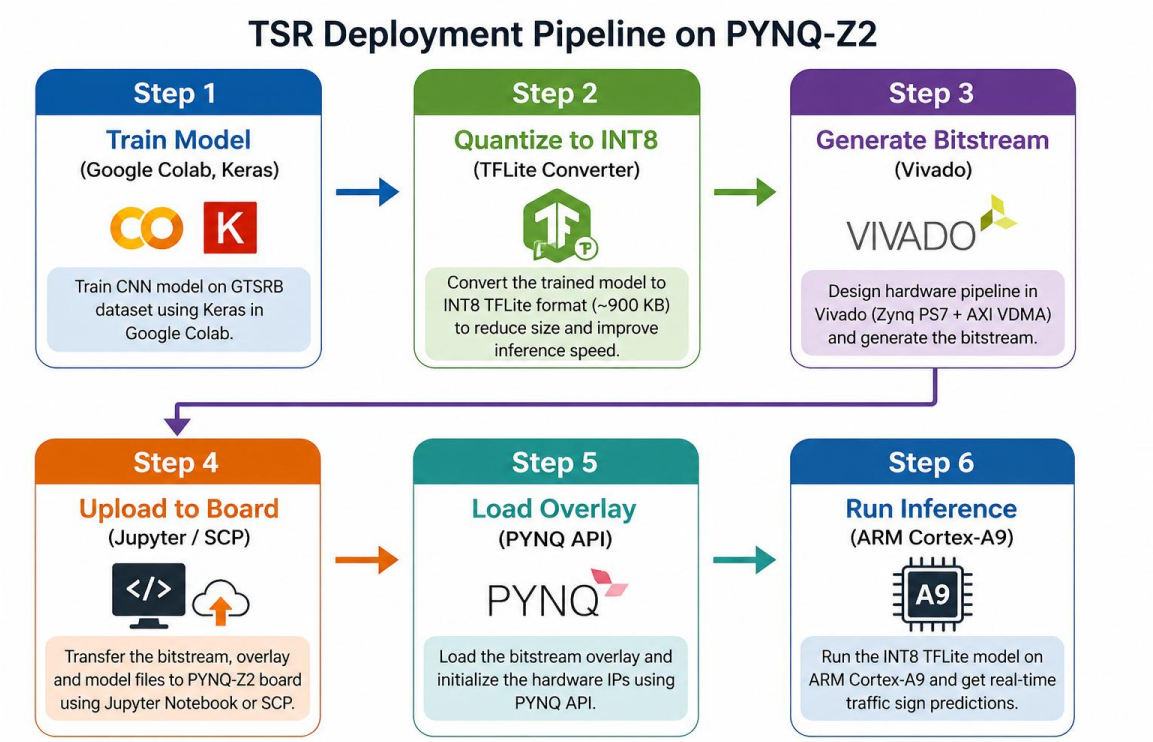

Fig 4: Deployment Workflow

9. Results & Verification

The performance of the model was rigorously evaluated both during the offline training phase and during live hardware inference on the PYNQ-Z2 board.

9.1 Training Performance

The model quickly converged, achieving over 90% validation accuracy within the first 10-15 epochs. The EarlyStopping callback ensured that the training halted at the optimal point before severe overfitting occurred. The final model yielded approximately 92% overall accuracy across the challenging 43-class dataset.

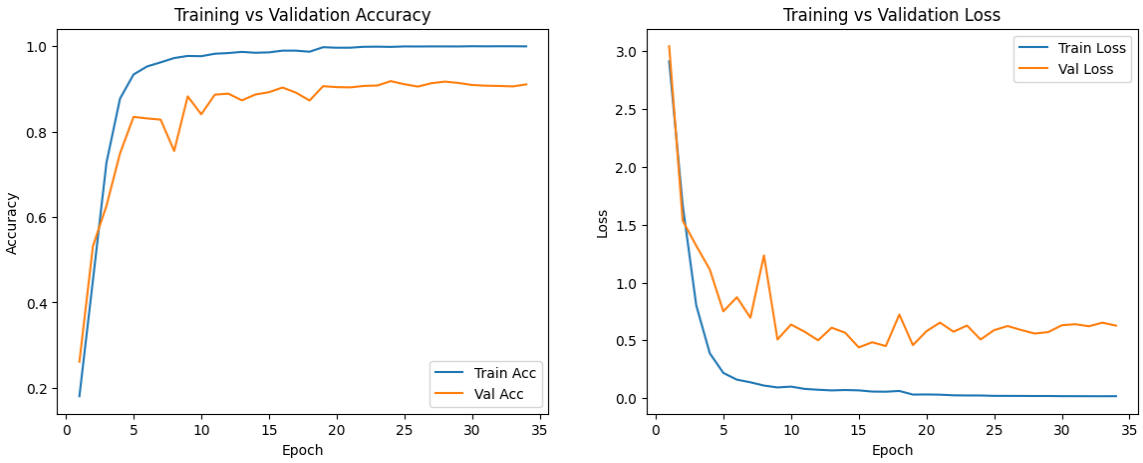

Fig 5: Training Accuracy & Loss Curves

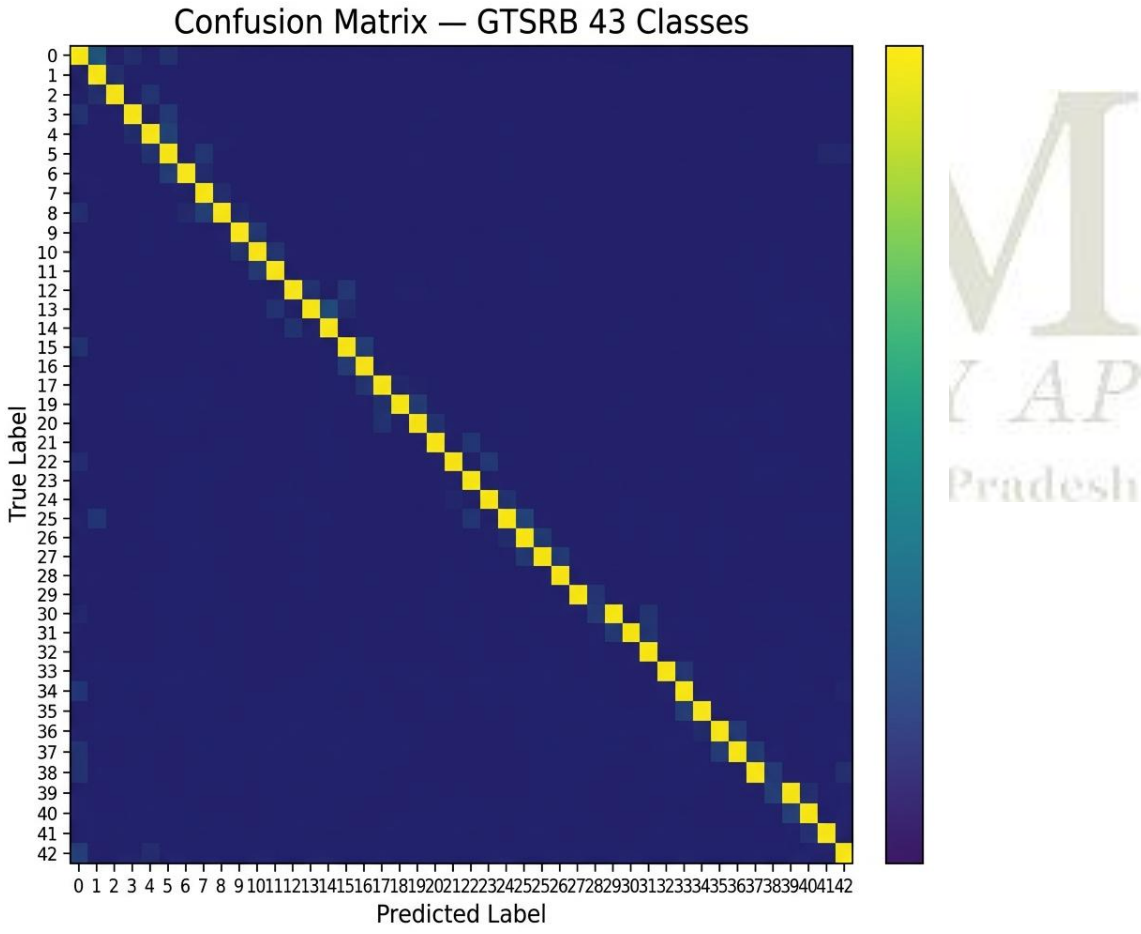

Fig 6: Confusion Matrix demonstrating robust classification across 43 Classes

9.2 Hardware Inference Results

When deployed on the PYNQ-Z2, the INT8 quantized TFLite model achieved an exceptional sub-30ms inference latency per frame on the ARM Cortex-A9 processor. This ensures a fluid

throughput of over 30 FPS, well within the requirements for real-time operation. During live tests, the model displayed high confidence (often >90% to 100%) when recognizing standard signs such as Speed Limits, Priority Roads, and Road Work warnings.

Fig 7: Hardware Prediction Output - Speed Limit

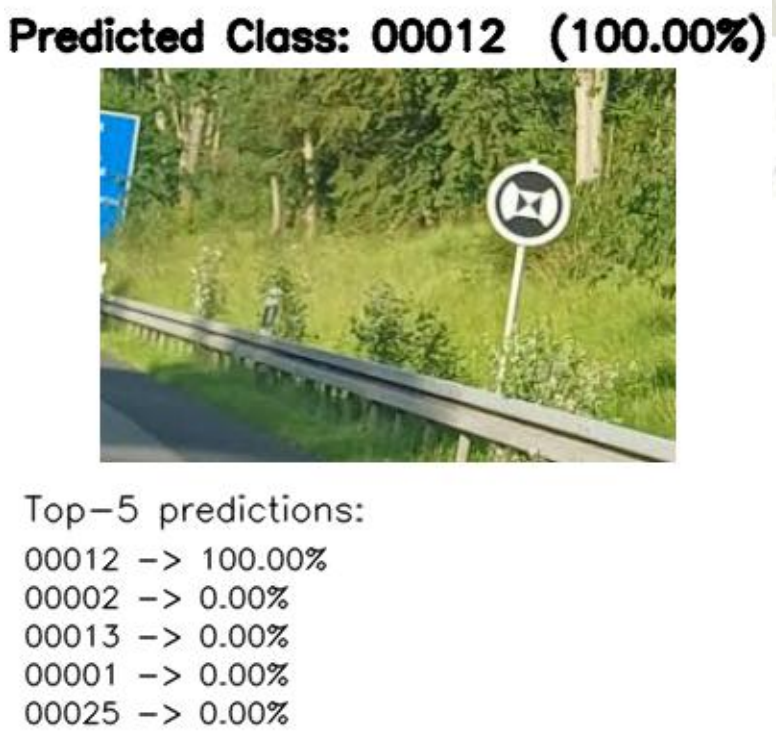

Fig 8: Hardware Prediction Output - Priority Road

10. Conclusion

This project successfully demonstrated the end-to-end design, optimization, and edge-deployment of a Traffic Sign Recognition system on the PYNQ-Z2 FPGA platform. By integrating a lightweight sequential CNN architecture with post-training INT8 quantization, the model footprint was aggressively reduced to ~900 KB without compromising its ~92% accuracy. Furthermore, the hardware-software co-design paradigm—leveraging Vivado for hardware-accelerated video streaming via AXI VDMA—allowed the embedded ARM processor to execute inference rapidly, achieving sub-30ms latency. The system stands as a highly efficient, low-power ADAS perception module suitable for real-time edge environments.